

运维进行时

互联网运维技术架构

Search Blog

个人简介



姓名：刘天斯

BLOG: <http://blog.liuts.com>

微博: <http://t.qq.com/yorkoliu>

<http://weibo.com/yorkoliu>

工作经历:

1、天涯社区(2005~2011)

2、腾讯(2011~今)

职务：架构师、高级系统工程师

发明专利：7 个

籍贯：海南琼海

Email、MSN:



个人著作:



技术交流QQ群:

106651609[1号] 满

106651547[2号] 满

37654606[3号] 满

8672312[4号] 满

158926355[5号] 满

251989396[6号]

汇总

招聘专栏

标签

留言

边栏

归档

星标日志

基于kubernetes构建Docker集群管理详解

刘天斯, 2014/12/22 21:41, Docker, 评论(20), 阅读(13176), Via 本站
原创

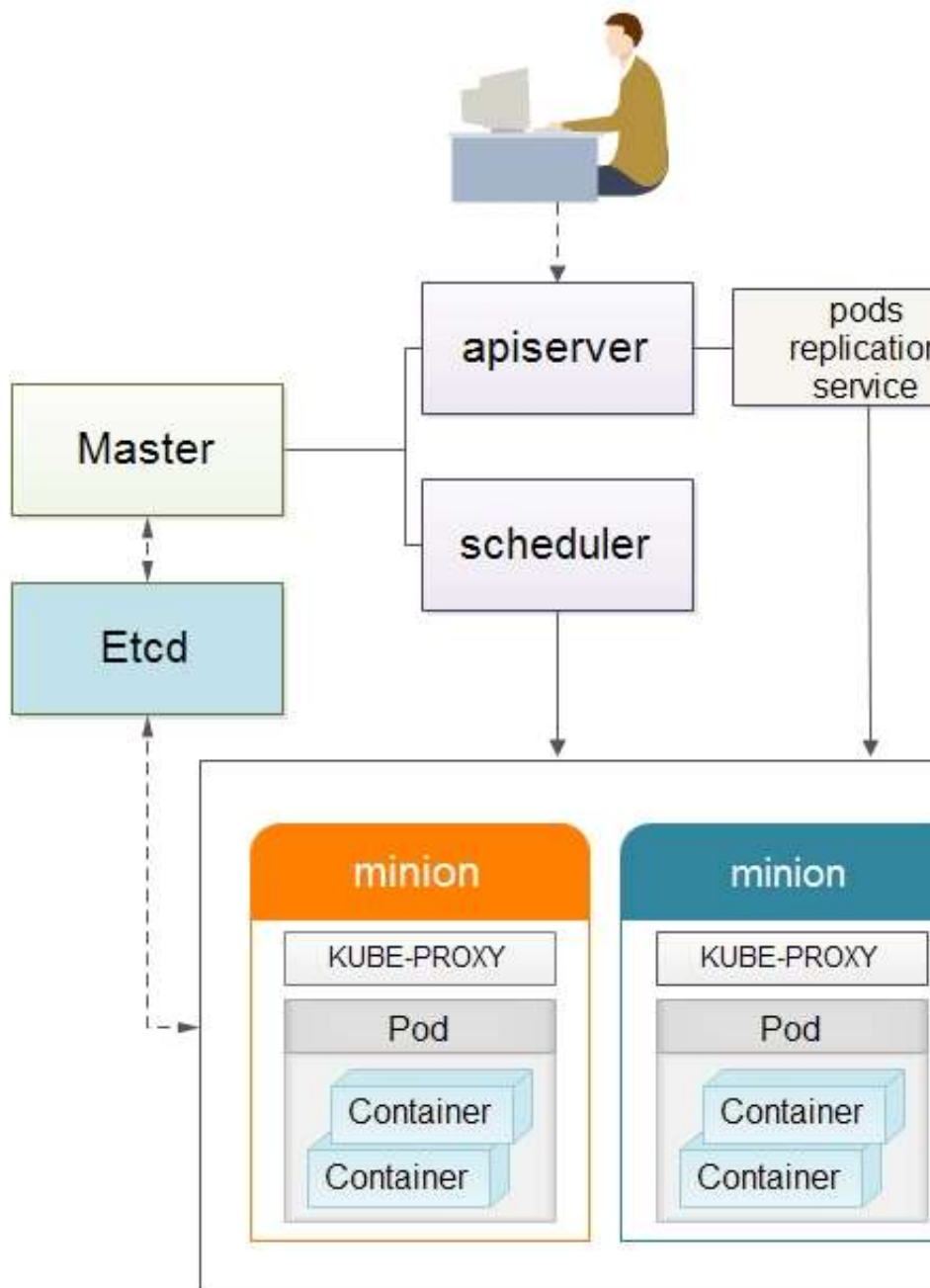
大 | 中 | 小

一、前言

Kubernetes 是Google开源的容器集群管理系统，基于Docker构建一个容器的调度服务，提供资源调度、均衡容灾、服务注册、动态扩缩容等功能套件，目前最新版本为0.6.2。本文介绍如何基于Centos7.0构建Kubernetes平台，在正式介绍之前，大家有必要先理解Kubernetes几个核心概念及其承担的功能。以下为Kubernetes的架构设计图：

203734755[资深]

分类

默认分类 [3] [RSS](#)Docker [3] [RSS](#)Linux [44] [RSS](#)Architecture [1] [RSS](#)SA [2] [RSS](#)rrdtool [1] [RSS](#)Windows [2] [RSS](#)Ubuntu [10] [RSS](#)MYSQL [17] [RSS](#)SQLSERVER [1] [RSS](#)Memcached [2] [RSS](#)MongoDB [2] [RSS](#)LVS [4] [RSS](#)DNS [6] [RSS](#)Apache [8] [RSS](#)Lighttpd [3] [RSS](#)Resin [1] [RSS](#)Haproxy [2] [RSS](#)Squid [1] [RSS](#)Varnish [3] [RSS](#)Nginx [2] [RSS](#)SHELL [14] [RSS](#)JAVA [4] [RSS](#)C/C++ [7] [RSS](#)PHP [11] [RSS](#)Python [38] [RSS](#)saltstack [1] [RSS](#)Perl [2] [RSS](#)Ajax [3] [RSS](#)CSS [2] [RSS](#)

1. Pods

在Kubernetes系统中，调度的最小颗粒不是单纯的容器，而是抽象成一个Pod，Pod是一个可以被创建、销毁、调度、管理的最小的部署单元。比如一个或一组容器。

2. Replication Controllers

Replication Controller是Kubernetes系统中最有用的功能，实现复制多个Pod副本，往往一个应用需要多个Pod来支撑，并且可以保证其复制的副本数，即使副本所调度分配的主宿机出现异常，通过Replication Controller可以保证在其它主宿机启用同等数量的Pod。Replication Controller可以通过repcn模板来创建多个Pod副本，同样也可以直接复制已存在Pod，需要通过Label selector来关联。

3、Services

Services是Kubernetes最外围的单元，通过虚拟一个访问IP及服务端口，可以访问我们定义好的Pod资源，目前的版本是通过iptables的nat转发来实现，转发的目标端口为Kube_proxy生成的随机

[Javascript \[6\]](#) [RSS](#)[Adode \[2\]](#) [RSS](#)[MooseFS \[3\]](#) [RSS](#)[Cfengine \[2\]](#) [RSS](#)[Splunk \[1\]](#) [RSS](#)[Func \[6\]](#) [RSS](#)[Qmail/Postfix \[2\]](#) [RSS](#)[RSA \[3\]](#) [RSS](#)[Hardware/Network \[1\]](#) [RSS](#)[TOOL/Software \[0\]](#) [RSS](#)[My Life \[10\]](#) [RSS](#)[我的著作 \[1\]](#) [RSS](#)

日历

< 2015 > < 7 >

乙未年（羊）

日	一	二	三	四	五	六
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

最新日志

[基于kubernetes...](#)[《python自动化运维...](#)[《python自动化运维...](#)[《Python自动化运维...](#)[【2014WOT深圳站讲...](#)[构建一个高可用及自动发现...](#)[Docker远程pyth...](#)[基于saltstack实...](#)[maptail实现实时、...](#)[基于Python实现的轻...](#)

最新评论

[直接复制你的json，用...](#)

端口，目前只提供GOOGLE云上的访问调度，如GCE。如果与我们自建的平台进行整合？请关注下篇《kubernetes与HECD架构的整合》文章。

4、Labels

Labels是用于区分Pod、Service、Replication Controller的key/value键值对，仅使用在Pod、Service、Replication Controller之间的关系识别，但对这些单元本身进行操作时得使用name标签。

5、Proxy

Proxy不但解决了同一主机相同服务端口冲突的问题，还提供了Service转发服务端口对外提供服务的能力，Proxy后端使用了随机、轮循负载均衡算法。

说说个人一点看法，目前Kubernetes 保持一周一小版本、一个月一大版本的节奏，迭代速度极快，同时也带来了不同版本操作方法的差异，另外官网文档更新速度相对滞后及欠缺，给初学者带来一定挑战。在上游接入层官方侧重点还放在GCE（Google Compute Engine）的对接优化，针对个人私有云还未推出一套可行的接入解决方案。在v0.5版本中才引用service代理转发的机制，且是通过iptables来实现，在高并发下性能令人担忧。但作者依然看好Kubernetes未来的发展，至少目前还未看到另外一个成体系、具备良好生态的平台，相信在V1.0时就会具备生产环境的服务支撑能力。

一、环境部署

1、平台版本说明

- 1) Centos7.0 OS
- 2) Kubernetes V0.6.2
- 3) etcd version 0.4.6
- 4) Docker version 1.3.2

2、平台环境说明

角色	主机名	IP
Master	SN2014-12-200	192.168.1.200
Etcd	SN2014-12-010	192.168.1.10
Minion	SN2014-12-201	192.168.1.201
Minion	SN2014-12-202	192.168.1.202

3、环境安装

直接复制你的json，用...

天斯哥，打扰了，最近在使...

vi Makefil...

scriptalert(...

大师，想了解一下你的个人...

大神什么时候给个json...

我也遇到了和 YUN 同...

请问ServerMap1...

天斯大神，你这个流程图我...

你好。请问下全局负载均衡...

博主：不清楚是否有人跟我...

mesos + mara...

#tail -fomsy...

部署是成功了，还整明白到...

大师，想请问下：“”中...

hi 正在读这本书在pa...

怎么实现自动检测宕机呢？

Dockerfile 中...

看是看官方文档吧 nms...

还可以

kubectl ge...

iptables DRO...

我在部署完 访问的时候 ...

我知道是什么问题了,是因...

其实我知道上午的主要问题...

老师我觉的这里是不是有点...

感觉 YAML格式要求很...

感觉YAML的格式要求很...

运行func * pin...

{func.slave....

天斯：昨天的出现的问题已...

运行出错No hosts...

求帮助为啥安装完后，点击...

访问包205错误。#ca...

书上是不是 没有下 创建...

Django+uwsgi...

1) 系统初始化工作（所有主机）

系统安装-选择[最小化安装]

引用

```
# yum -y install wget ntpdate bind-utils
```

```
# wget
```

```
http://mirror.centos.org/centos/7/extras/x86_64/Packages/epel-  
release-7-2.noarch.rpm
```

```
# yum update
```

CentOS 7.0默认使用的是firewall作为防火墙，这里改为iptables防火墙（熟悉度更高，非必须）。

1.1、关闭firewall:

引用

```
# systemctl stop firewalld.service #停止firewall
```

```
# systemctl disable firewalld.service #禁止firewall开机启动
```

1.2、安装iptables防火墙

引用

```
# yum install iptables-services #安装
```

```
# systemctl start iptables.service #最后重启防火墙使配置生效
```

```
# systemctl enable iptables.service #设置防火墙开机启动
```

2) 安装Etcd（192.168.1.10主机）

引用

请教一下。我在ubunt...

已经买下

搞来搞去。都是 入库失败...

跨机器的docker内部...

请问下DNS查询时间，链...

天斯大人: Python...

天斯兄....小弟在测试...

好书，博主的博客已经关注...

您好，请问nginx p...

天斯兄，麻烦看看是什么问...

天斯兄，modules目...

我方向可能找错了。E01...

天斯，这个关于json的...

请教下，我用自己的容器测...

天斯你好：还是这个问题我...

天斯哥...这个json...

天斯哥....想问问你....

天斯你好：请我下面这个报...

api模式的怎么运行啊

对了，新的主机什么的 主...

安装部署好了，添加模块还...

好。。。书上和源代码 运...

看到了 数据库连接重新配...

安装好了，，omserv...

python是不是2.7...

mysql搞好了，自己建...

mysql的版本有没有要...

手册里面没有写到mysq...

OMServer里面的O...

SQL导入这步要怎么搞啊...

OMServer.sql...

终于安装成功了，是系统自...

好！为什么安装uwsg...

请教一个问题第一章1.3...

我执行的就是本文中的命令...

```
# mkdir -p /home/install && cd /home/install
# wget
https://github.com/coreos/etcd/releases/download/v0.4.6/etcd-
v0.4.6-linux-amd64.tar.gz
# tar -zxvf etcd-v0.4.6-linux-amd64.tar.gz
# cd etcd-v0.4.6-linux-amd64
# cp etcd* /bin/
# /bin/etcd -version
etcd version 0.4.6
```

启动服务etcd服务，如有提供第三方管理需求，另需在启动参数中添加"-cors='*'"参数。

引用

```
# mkdir /data/etcd
# /bin/etcd -name etcdserver -peer-addr 192.168.1.10:7001 -
addr 192.168.1.10:4001 -data-dir /data/etcd -peer-bind-addr
0.0.0.0:7001 -bind-addr 0.0.0.0:4001 &
```

配置etcd服务防火墙，其中4001为服务端口，7001为集群数据交互端口。

引用

```
# iptables -I INPUT -s 192.168.1.0/24 -p tcp -dport 4001 -j
ACCEPT
# iptables -I INPUT -s 192.168.1.0/24 -p tcp -dport 7001 -j
ACCEPT
```

couldnt get ...

我下了最新的0.7.1, ...

好吧。。。wsgi.p...

嗯, 尝试着改了下能pin...

但demo.domain...

我在搭建之后一直到`启动...

IPy 貌似只能支持24...

为神马看完还是觉得弄不好...

链 接

同事好友

蒙其·B·宾森的博客

嘻嘻9_9 BLOG

Sam in Tianya

粗茶淡饭

Vimer的程序世界

chauvet's blog

MinUnix系统运维

Nornor's Blog

系统运维

IT好友

BMForum

回忆未来[张宴]

Jean — 记录成长历程

鱼常游而忘飞

dba blog

恋上E人

网海过客

unixhot

灰太狼的blog

抚琴煮酒

selboo's blog

懒人运维

K.I.S.S. — 简单哲学

badb0y's blog

young for you

ThinkSOA

IT工地

晓辉工作室

Queryer

tickecho老吴

story的天空

3) 安装Kubernetes (涉及所有Master、Minion主机)

通过yum源方式安装, 默认将安装etcd, docker, and cadvisor相关包。

引用

```
# curl https://copr.fedoraproject.org/coprs/eparis/kubernetes-epel-7/repo/epel-7/eparis-kubernetes-epel-7-epel-7.repo -o /etc/yum.repos.d/eparis-kubernetes-epel-7-epel-7.repo
#yum -y install kubernetes
```

升级至v0.6.2, 覆盖bin文件即可, 方法如下:

引用

```
# mkdir -p /home/install && cd /home/install
# wget
https://github.com/GoogleCloudPlatform/kubernetes/releases/download/v0.6.2/kubernetes.tar.gz
# tar -zxvf kubernetes.tar.gz
# tar -zxvf kubernetes/server/kubernetes-server-linux-amd64.tar.gz
# cp kubernetes/server/bin/kube* /usr/bin
```

校验安装结果, 出版以下信息说明安装正常。

引用

```
[root@SN2014-12-200 bin]# /usr/bin/kubectl version
Client Version: version.Info{Major:"0", Minor:"6+",
GitVersion:"v0.6.2",
GitCommit:"729fde276613eedcd99ecf5b93f095b8deb64eb4",
GitTreeState:"clean"}
```

[英才and栋梁'S Blog](#)[疯子的视界](#)[雨林木风-高进波](#)[ISADBA|FH.CN](#)[老万-不知所谓](#)[Vi` blog](#)[linxun's blog](#)[运维人生](#)[kindle's blog](#)[e2fsc's blog](#)[Phoenix的实验室](#)[SYSTEMADMIN](#)[安静的角落！](#)[专注linux领域](#)[Sudu's Blog](#)[Laird-SA](#)[鸪鸪天BLOG](#)[darkmi'blog](#)[运维日志](#)[阿熊的窝](#)[陋室博客](#)[51运维管理社区](#)[青年怪客](#)[叶茂盛](#)[Pw](#)[飛飛's Blog](#)[Virtual World](#)[centos中文站](#)[未来往事](#)[IT自学网](#)[峰云，就她了。](#)[linux菜园子](#)[51运维网](#)[Linux中国](#)[tshare365](#)[Dig All Possible](#)

开源项目

[天涯LVS管理系统](#)[天涯服务器管理系统\(C/S版\)](#)[天涯服务器管理系统\(手机版\)](#)[在线图片处理平台](#)[服务器机柜模拟图平台](#)[SDR-Linux主机集中管理](#)[Varnish缓存推送平台V1.0](#)

网址收藏

Server Version: &version.Info{Major:"0", Minor:"6+",

GitVersion:"v0.6.2",

GitCommit:"729fde276613eedcd99ecf5b93f095b8deb64eb4",

GitTreeState:"clean"}

4) Kubernetes配置（仅Master主机）

master运行三个组件,包括apiserver、scheduler、controller-manager，相关配置项也只涉及这三块。

4.1、【/etc/kubernetes/config】

view plain print ?

```
1. # Comma seperated list of nodes in the etcd cluster
2. KUBE_ETCD_SERVERS="--
   etcd_servers=http://192.168.1.10:4001"
3.
4. # logging to stderr means we get it in the systemd
   journal
5. KUBE_LOGTOSTDERR="--logtostderr=true"
6.
7. # journal message level, 0 is debug
8. KUBE_LOG_LEVEL="--v=0"
9.
10. # Should this cluster be allowed to run privileged d
    ocker containers
11. KUBE_ALLOW_PRIV="--allow_privileged=false"
```

4.2、【/etc/kubernetes/apiserver】

view plain print ?

```
1. # The address on the local server to listen to.
2. KUBE_API_ADDRESS="--address=0.0.0.0"
3.
4. # The port on the local server to listen on.
5. KUBE_API_PORT="--port=8080"
6.
7. # How the replication controller and scheduler find
   the kube-apiserver
8. KUBE_MASTER="--master=192.168.1.200:8080"
9.
10. # Port minions listen on
11. KUBELET_PORT="--kubelet_port=10250"
12.
13. # Address range to use for services
14. KUBE_SERVICE_ADDRESSES="--
   portal_net=10.254.0.0/16"
15.
16. # Add you own!
17. KUBE_API_ARGS=""
```

4.3、【/etc/kubernetes/controller-manager】

view plain print ?

[SED单行脚本快速参考](#)[C++ 标准库](#)[python在线命令行](#)[Python资源索引](#)[htaccess在线生成](#)[在线检查HTTP HEAD](#)[有奖积分小游戏](#)[python代码搜索引擎](#)[php作者之PHP Performance](#)[Google AdSense](#)[CSS-BUTTON](#)[在线绘图](#)[doit时间管理](#)[python模块集中营](#)

归 档

[2015/07](#)[2015/06](#)[2015/05](#)[2015/04](#)[2015/03](#)

其 他

[登入](#)[注册](#)[申请链接](#)[RSS: 日志 | 评论](#)[编码: UTF-8](#)[XHTML 1.0](#)

```
1. # Comma seperated list of minions
2. KUBELET_ADDRESSES="--
   machines= 192.168.1.201,192.168.1.202"
3.
4. # Add you own!
5. KUBE_CONTROLLER_MANAGER_ARGS=""
```

4.4、【/etc/kubernetes/scheduler】

```
view plain print ?
1. # Add your own!
2. KUBE_SCHEDULER_ARGS=""
```

启动master侧相关服务

引用

```
# systemctl daemon-reload
# systemctl start kube-apiserver.service kube-controller-
manager.service kube-scheduler.service
# systemctl enable kube-apiserver.service kube-controller-
manager.service kube-scheduler.service
```

5) Kubernetes配置（仅minion主机）

minion运行两个组件,包括kubelet、proxy，相关配置项也只涉及这两块。

Docker启动脚本更新

```
# vi /etc/sysconfig/docker
```

添加: -H tcp://0.0.0.0:2375，最终配置如下，以便以后提供远程API维护。

```
OPTIONS=--selinux-enabled -H tcp://0.0.0.0:2375 -H fd://
```

修改minion防火墙配置，通常master找不到minion主机多半是由于端口没有连通。

```
iptables -I INPUT -s 192.168.1.200 -p tcp --dport 10250 -j
ACCEPT
```

修改kubernetes minion端配置，以192.168.1.201主机为例，其它minion主机同理。

5.1、【/etc/kubernetes/config】

```
view plain print ?
1.  # Comma seperated list of nodes in the etcd cluster
2.  KUBE_ETCD_SERVERS="--
    etcd_servers=http://192.168.1.10:4001"
3.
4.  # logging to stderr means we get it in the systemd
    journal
5.  KUBE_LOGTOSTDERR="--logtostderr=true"
6.
7.  # journal message level, 0 is debug
8.  KUBE_LOG_LEVEL="--v=0"
9.
10. # Should this cluster be allowed to run privileged d
    ocker containers
11. KUBE_ALLOW_PRIV="--allow_privileged=false"
```

5.2、【/etc/kubernetes/kubelet】

```
view plain print ?
1.  ###
2.  # kubernetes kubelet (minion) config
3.
4.  # The address for the info server to serve on (set
    to 0.0.0.0 or "" for all interfaces)
5.  KUBELET_ADDRESS="--address=0.0.0.0"
6.
7.  # The port for the info server to serve on
8.  KUBELET_PORT="--port=10250"
9.
10. # You may leave this blank to use the actual hostna
    me
11. KUBELET_HOSTNAME="--
    hostname_override=192.168.1.201"
12.
13. # Add your own!
14. KUBELET_ARGS=""
```

5.3、【/etc/kubernetes/proxy】

```
view plain print ?
1.  KUBE_PROXY_ARGS=""
```

启动kubernetes服务

引用

```
# systemctl daemon-reload
# systemctl enable docker.service kubelet.service kube-proxy.service
# systemctl start docker.service kubelet.service kube-proxy.service
```

3、校验安装(在master主机操作，或可访问master主机8080端口的client api主机)

1) kubernetes常用命令

引用

```
# kubectl get minions    #查看minion主机
# kubectl get pods      #查看pods清单
# kubectl get services 或 kubectl get services -o json    #查看
service清单
# kubectl get replicationControllers    #查看replicationControllers清单
# for i in `kubectl get pod|tail -n +2|awk '{print $1}'`; do kubectl
delete pod $i; done    #删除所有pods
```

或者通过Server api for REST方式（推荐，及时性更高）：

引用

```
# curl -s -L http://192.168.1.200:8080/api/v1beta1/version |
python -mjson.tool    #查看kubernetes版本
# curl -s -L http://192.168.1.200:8080/api/v1beta1/pods | python -
mjson.tool    #查看pods清单
# curl -s -L
http://192.168.1.200:8080/api/v1beta1/replicationControllers |
python -mjson.tool    #查看replicationControllers清单
# curl -s -L http://192.168.1.200:8080/api/v1beta1/minions |
python -m json.tool    #查看minion主机
# curl -s -L http://192.168.1.200:8080/api/v1beta1/services |
python -m json.tool    #查看service清单
```

注：在新版kubernetes中，所有的操作命令都整合至kubectl，包括

kubecfg、kubectl.sh、kubecfg.sh等

2) 创建测试pod单元

```
# /home/kubermange/pods && cd /home/kubermange/pods
```

```
# vi apache-pod.json
```

```
view plain print ?
1.  {
2.    "id": "fedoraapache",
3.    "kind": "Pod",
4.    "apiVersion": "v1beta1",
5.    "desiredState": {
6.      "manifest": {
7.        "version": "v1beta1",
8.        "id": "fedoraapache",
9.        "containers": [{
10.         "name": "fedoraapache",
11.         "image": "fedora/apache",
12.         "ports": [{
13.           "containerPort": 80,
14.           "hostPort": 8080
15.         }]
16.       }]
17.     },
18.     "labels": {
19.       "name": "fedoraapache"
20.     }
21.   }
22. }
```

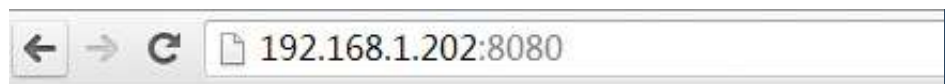
```
# kubectl create -f apache-pod.json
```

```
# kubectl get pod
```

引用

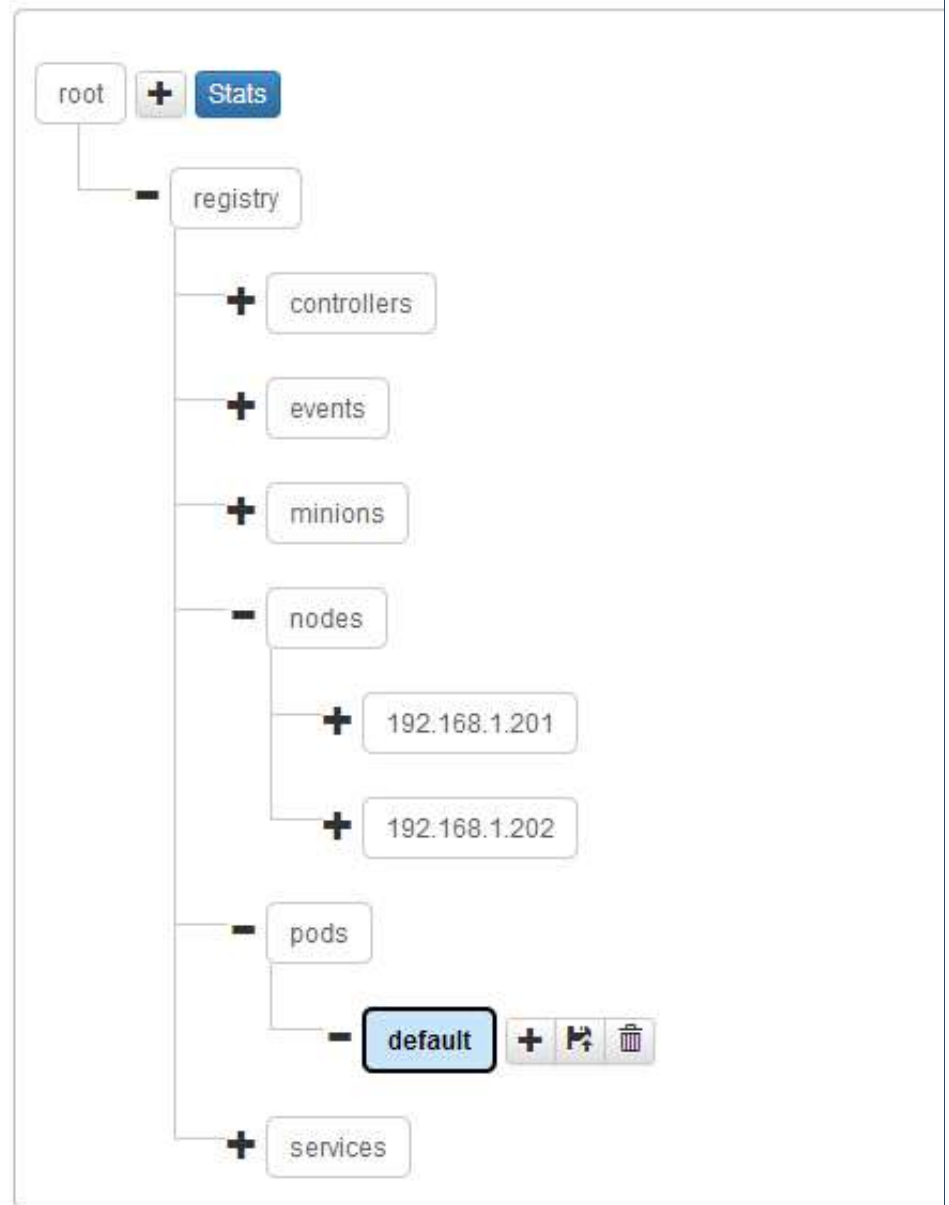
NAME	IMAGE(S)	HOST	LABELS
STATUS			
fedoraapache	fedora/apache		
192.168.1.202/	name=fedoraapache	Running	

启动浏览器访问<http://192.168.1.202:8080/>，对应的服务端口切记在iptables中已添加。效果图如下：

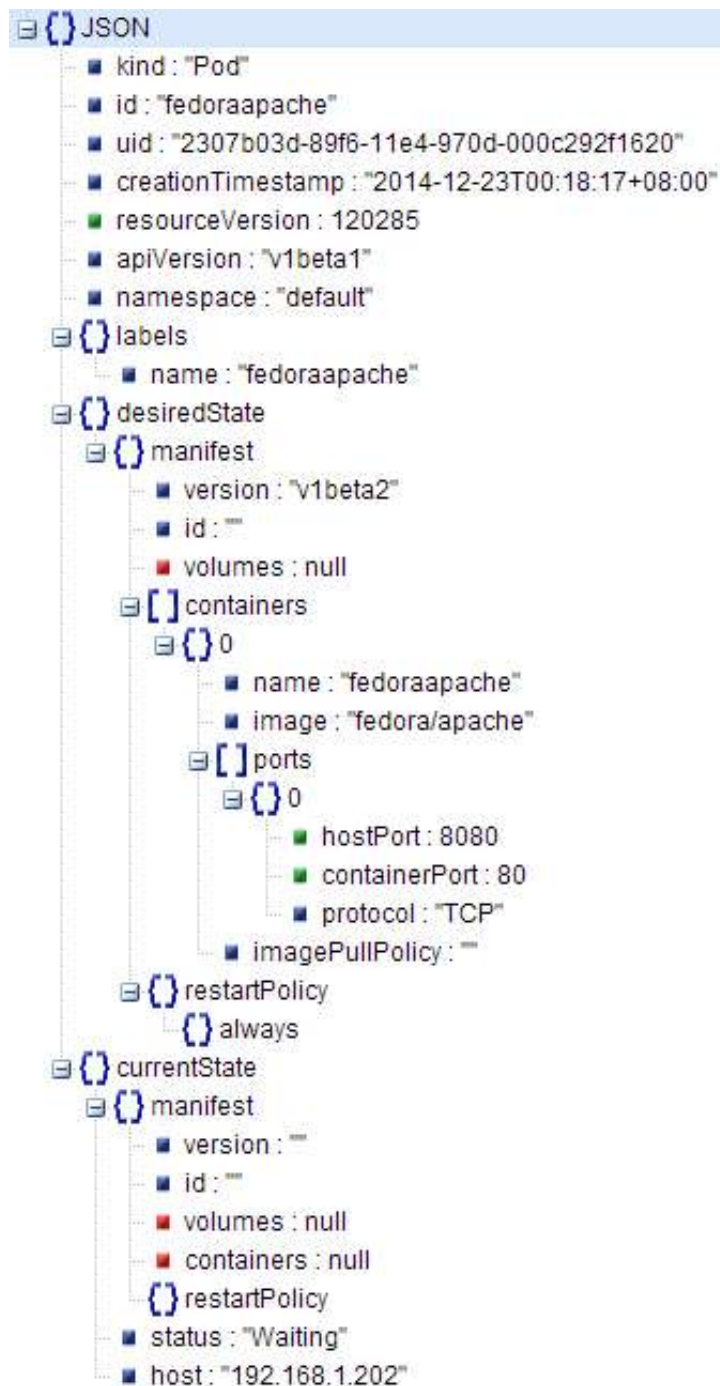


Apache

观察kubernetes在etcd中的数据存储结构



观察单个pods的数据存储结构，以json的格式存储。



二、实战操作

任务：通过Kubernetes创建一个LNMP架构的服务集群，以及观察其负载均衡，涉及镜像“yorko/webserver”已经push至registry.hub.docker.com，大家可以通过“docker pull yorko/webserver”下载。

引用

```
# mkdir -p /home/kubermange/replication && mkdir -p
/home/kubermange/service
# cd /home/kubermange/replication
```

1、创建一个replication，本例直接在replication模板中创建pod并复制，也可独立创建pod再通过replication来复制。

【replication/lnmp-replication.json】

```
view plain print ?
1.  {
2.    "id": "webserverController",
3.    "kind": "ReplicationController",
4.    "apiVersion": "v1beta1",
5.    "labels": {"name": "webserver"},
6.    "desiredState": {
7.      "replicas": 2,
8.      "replicaSelector": {"name": "webserver_pod"},
9.      "podTemplate": {
10.        "desiredState": {
11.          "manifest": {
12.            "version": "v1beta1",
13.            "id": "webserver",
14.            "volumes": [
15.              {"name": "httpconf", "source":
16.                {"hostDir": {"path": "/etc/httpd/conf"}}},
17.              {"name": "httpconfd", "source":
18.                {"hostDir": {"path": "/etc/httpd/conf.d"}}},
19.              {"name": "httproot", "source":
20.                {"hostDir": {"path": "/data"}}}
21.            ],
22.            "containers": [{
23.              "name": "webserver",
24.              "image": "yorko/webserver",
25.              "command": ["/bin/sh", "-
26.                c", "/usr/bin/supervisord -
27.                c /etc/supervisord.conf"],
28.              "volumeMounts": [
29.                {"name": "httpconf", "mountPath": "/et
30.                  c/httpd/conf"},
31.                {"name": "httpconfd", "mountPath": "/e
32.                  tc/httpd/conf.d"},
33.                {"name": "httproot", "mountPath": "/da
34.                  ta"}
35.              ],
36.              "cpu": 100,
37.              "memory": 50000000,
38.              "ports": [{
39.                "containerPort": 80,
40.              }, {
41.                "containerPort": 22,
42.              }]
43.            }]
44.          }
45.        },
46.        "labels": {"name": "webserver_pod"},
47.      },
48.    ]
49.  }
```

执行创建命令

```
#kubectl create -f lnmp-replication.json
```

观察生成的pod副本清单：

```
[root@SN2014-12-200 replication]# kubectl get pod
```

引用

NAME			
IMAGE(S)	HOST	LABELS	STATUS
84150ab7-89f8-11e4-970d-000c292f1620	yorko/webserver		
192.168.1.202/	name=webserver_pod	Running	
84154ed5-89f8-11e4-970d-000c292f1620	yorko/webserver		
192.168.1.201/	name=webserver_pod	Running	
840beb1b-89f8-11e4-970d-000c292f1620	yorko/webserver		
192.168.1.202/	name=webserver_pod	Running	
84152d93-89f8-11e4-970d-000c292f1620	yorko/webserver		
192.168.1.202/	name=webserver_pod	Running	
840db120-89f8-11e4-970d-000c292f1620	yorko/webserver		
192.168.1.201/	name=webserver_pod	Running	
8413b4f3-89f8-11e4-970d-000c292f1620	yorko/webserver		
192.168.1.201/	name=webserver_pod	Running	

2、创建一个service，通过selector指定 "name": "webserver_pod"与 pods关联。

【service/lnmp-service.json】

```
view plain print ?
1.  {
2.    "id": "webserver",
3.    "kind": "Service",
4.    "apiVersion": "v1beta1",
5.    "selector": {
6.      "name": "webserver_pod",
7.    },
8.    "protocol": "TCP",
9.    "containerPort": 80,
10.   "port": 8080
11. }
```

执行创建命令：

```
# kubectl create -f lnmp-service.json
```

登录minion主机（192.168.1.201），查询主宿机生成的iptables转发规则（最后一行）

```
# iptables -nvL -t nat
```

引用

Chain KUBE-PROXY (2 references)

pkts	bytes	target	prot	opt	in	out	source
------	-------	--------	------	-----	----	-----	--------

pkts	bytes	target	prot	opt	in	out	source
------	-------	--------	------	-----	----	-----	--------

2	120	REDIRECT	tcp	-	*	*	
---	-----	----------	-----	---	---	---	--

0.0.0.0/0		10.254.102.162					/* kubernetes */ tcp
-----------	--	----------------	--	--	--	--	----------------------

dpt:443	redir ports	47700					
---------	-------------	-------	--	--	--	--	--

1	60	REDIRECT	tcp	-	*	*	
---	----	----------	-----	---	---	---	--

0.0.0.0/0		10.254.28.74					/* kubernetes-ro */ tcp
-----------	--	--------------	--	--	--	--	-------------------------

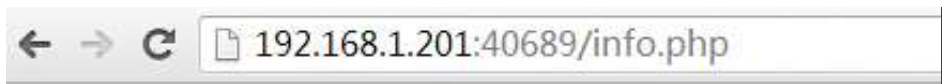
dpt:80	redir ports	60099					
--------	-------------	-------	--	--	--	--	--

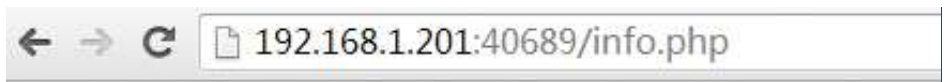
0	0	REDIRECT	tcp	--	*	*	
---	---	----------	-----	----	---	---	--

0.0.0.0/0		10.254.216.51					/* webserver */ tcp
-----------	--	---------------	--	--	--	--	---------------------

dpt:8080	redir ports	40689					
----------	-------------	-------	--	--	--	--	--

访问测试，<http://192.168.1.201:40689/info.php>，刷新浏览器发现proxy后端的变化，默认为随机轮循算法。





三、测试过程

1、pods自动复制、销毁测试，观察kubernetes自动保持副本数（6份）

删除replicationcontrollers中一个副本fedoraapache

```
[root@SN2014-12-200 pods]# kubectl delete pods fedoraapache
I1219 23:59:39.305730 9516 restclient.go:133] Waiting for
completion of operation 142530
fedoraapache
```

引用

```
[root@SN2014-12-200 pods]# kubectl get pods
```

NAME

IMAGE(S)

HOST

LABELS

STATUS

```
5d70892e-8794-11e4-970d-000c292f1620 fedora/apache
192.168.1.201/ name=fedoraapache Running
5d715e56-8794-11e4-970d-000c292f1620 fedora/apache
192.168.1.202/ name=fedoraapache Running
5d717f8d-8794-11e4-970d-000c292f1620 fedora/apache
192.168.1.202/ name=fedoraapache Running
5d71c584-8794-11e4-970d-000c292f1620 fedora/apache
192.168.1.201/ name=fedoraapache Running
5d71a494-8794-11e4-970d-000c292f1620 fedora/apache
192.168.1.202/ name=fedoraapache Running
```

#自动生成一个副本，保持6份的效果

引用

```
[root@SN2014-12-200 pods]# kubectl get pods
NAME
IMAGE(S)          HOST          LABELS          STATUS
5d717f8d-8794-11e4-970d-000c292f1620 fedora/apache
192.168.1.202/    name=fedoraapache Running
5d71c584-8794-11e4-970d-000c292f1620 fedora/apache
192.168.1.201/    name=fedoraapache Running
5d71a494-8794-11e4-970d-000c292f1620 fedora/apache
192.168.1.202/    name=fedoraapache Running
2a8fb993-8798-11e4-970d-000c292f1620 fedora/apache
192.168.1.201/    name=fedoraapache Running
5d70892e-8794-11e4-970d-000c292f1620 fedora/apache
192.168.1.201/    name=fedoraapache Running
5d715e56-8794-11e4-970d-000c292f1620 fedora/apache
192.168.1.202/    name=fedoraapache Running
```

2、测试不同角色模块中的hostPort

1) pod中hostPort为空，而replicationcontrollers为指定端口，则异常；两侧都指定端口，相同或不同时都异常；pod的hostport为指定，另replicationcon为空，则正常；pod的hostport为空，另replicationcon为空，则正常；结论是在replicationcontrollers场景不能指定hostport，否则异常，待持续测试。

2) 结论：在replicationcontronllers.json中，"replicaSelector":

```
{"name": "webserver_pod"}要与"labels": {"name":  
"webserver_pod"}以及service中的"selector": {"name":  
"webserver_pod"} 保持一致;
```

请关注下篇《kubernetes与HECD架构的整合》，近期推出。

参考文献:

https://github.com/GoogleCloudPlatform/kubernetes/blob/master/docs/getting-started-guides/fedora/fedora_manual_config.md

<https://github.com/GoogleCloudPlatform/kubernetes/blob/master/DESIGN.md>

<http://www.infoq.com/cn/articles/Kubernetes-system-architecture-introduction>

转载请注明来源 <http://blog.liuts.com/post/247/>

Tags: docker , kubernetes , 容器管理 , 刘天斯

构建一个高可用及自动发现

GB1973

2015/07/10 14:41

直接复制你的json，用kubectl create -f <json文件>总是提示：error: no objects passed to create
==>可能你里面大小写问题.我已开始也这个提示,后来把"kind": "pod",改为"kind": "Pod" 就好了

icecream

2015/07/08 11:59

直接复制你的json，用kubectl create -f <json文件>总是提示：error: no objects passed to create

Jack003

2015/06/03 16:19

大师，想了解一下你的个人网站的配置和每天的访问情况是多少？谢谢，我也在做自己的个人网站。

cuidong008

2015/06/01 15:22

大神什么时候给个json文件编写的手册，很期待啊

roadog

2015/05/11 15:16

我也遇到了和 YUN 同样的问题即 针对一个 rc 对应的pod 进行service 的导出用 curl 在 pod 所在node上 访问这个service 是可以的,但是在别的节点 虽然看到iptables 里已经做了映射 ,但是访问这个service 就会访问不到查询对应的 proxy logAccepted TCP connection from 10.1.52.0:60668 to 192.168.5.26:42787Dial failed: dial tcp 10.1.5.2:80: i/o timeout Failed to connect to balancer: failed to connect to an endpoint.

[lajie008](#)

2015/05/07 16:59

天斯大神，你这个流程图我很喜欢，请问是用的什么软件画的呢？

[ceshi](#)

2015/04/29 10:57

博主：不清楚是否有人跟我碰到同样的问题，我按照步骤装环境，我没有安装防火墙，kubernetes版本为最新0.9，装好以后minion一直是NotReady的状态，查log，可以看到下面出错的信息，不知道怎么解决这个问题，求解答Can't create rw endpoints: 501: All the given peers are not reachable (Tried to connect to each peer twice and failed) [0] watch was unable to retrieve the current index for the provided key ("/registry/controllers"): 501: All the given peers are not reachable (Tried to connect to each peer twice and failed) [0]

[问心](#) 

2015/04/25 13:40

mesos + marathon，是不是架构比较简单，不需要 kubernetes+HECD

[haha](#)

2015/04/23 09:42

大师，想请问下：“”中国运维领域偶像级专家，基于python开发的集群自动化操作工具—yorauto，在腾讯各大事业 群广泛使用“”这个工具广泛使用在腾讯哪里？举几个具体的例子，我想看看

[dannyzhang](#) 

2015/03/20 17:40

[root@localhost kubernetes]# kubect| get minions F0320
15:49:55.634705 24544 get.go:75] 501: All the given peers are not reachable (Tried to connect to each peer twice and failed) [0]请问报出来这个错误是为什么？

[liuyt](#) 

2015/02/25 16:04

请教一下。我在ubuntu上单物理机安装Kubernetes。环境是ubuntu 14.04，Kubernetes 0.11.0，etcd 2.0.3。安装完成后 查询如 version, pods, replicationControllers ,services, 都正常但是minions异常，显示"reason": "Node health check failed: kubelet /healthz endpoint returns not ok", "status": "None查看服务kubelet和kube-proxy是启动着的。端口10250也是被kubelet打开着的。想知道这个health是怎么检测出来的，以及大概出错的可能原因

[hiker](#)  

2015/01/23 17:01

天斯大人: Python 你用的是什么 IDE。我也想学习。

刘天斯 回复于 2015/01/23 17:30

Notepad++、Sublime Text2

[汤佳兴](#)

2015/01/12 14:19

天斯兄....小弟在测试的过程中，感觉这个service好像只能包含同Minion中的Container才能访问，是这样吗？？因为我在创建replication的时候指定了数量为2，创建出来刚好分布在不同的Minion上，我创建完service后测试，必须得输入两个Minion端的IP才能访问...这样的话，是不是前端最好弄个代理会好点

....

刘天斯 回复于 **2015/01/12 14:30**

是的，请关注下篇《kubernetes与HECD架构的整合》

YUN

2015/01/07 15:20

我方向可能找错了。E0107 15:17:53.969562 23004 proxier.go:77] Dial failed: dial tcp 172.17.0.12:8161: no route to hostE0107 15:17:54.969934 23004 proxier.go:77] Dial failed: dial tcp 172.17.0.12:8161: i/o timeout我的意思是，service启了以后，我发现service可能在master上，也可能在minions上。不管在哪，proxy的日志里，显示它不能访问另一台机器的容器地址。也即，当前service永远是拿自己的容器提供的服务，另一台它访问不了。

Yun  

2015/01/06 11:15

请教下，我用自己的容器测试的。2台机器，master启了kubelet和proxy。分别访问master和minion上的应用都是好的，但通过service访问，内容一直是master上容器返回，master日志/var/log/upstart/kube-proxy.log里有如下信息。E0105 12:50:21.908164 17848 proxier.go:77] Dial failed: dial tcp 172.17.0.205:8161: no route to host 问题来了，docker0的ip一样，而现在proxy想访问minion容器里的地址，怎么解决。

刘天斯 回复于 **2015/01/06 21:23**

怎么是docker0的IP? "172.17.0.205:8161: no route to host"

Tom

2014/12/31 17:48

天斯哥...这个json文件中有哪些属性是可以用的啊?? 有没有一个详细的说明文档啊..我找了半天都没找到...天斯哥..还望指定啊

刘天斯 回复于 **2014/12/31 19:20**

目前这块文档是欠缺的。

Tom

2014/12/31 16:29

天斯哥....想问问你..这个json文件必须得自己写吗?? 里面很多选项我都看不明白啊...还有，如果用json文件的，那怎么样用它来指定我内部的docker仓库呢?? 因为直接从外网下镜像感觉很慢，而且有时还下不下来....感谢指导。

刘天斯 回复于 **2014/12/31 19:20**

需要自己写。

wangcongxiang

2014/12/25 13:23

我执行的就是本文中的命令，也是用的.json文件，报的这个错确实奇怪，不过我后来换了一个0.3时候测试的.json文件就不报错了。

刘天斯 回复于 **2014/12/25 14:40**

看来还是版本产生的差异。

wangcongxiang  

2014/12/24 21:06

couldn't get version/kind: error converting YAML to JSON: yaml: line 15: did not find expected ',' or '}'

刘天斯 回复于 **2014/12/24 22:59**

本文使用JSON格式，非YAML。

wangcongxiang  

2014/12/24 21:04

我下了最新的0.7.1，运行kubectI create -f apache-pod.json报错

刘天斯 回复于 **2014/12/24 23:00**

0.7.1是昨天出的版本，不清楚update项，使用V0.6.2吧。

分页: 1/1  **1** 

发表评论

昵称

网址

电邮

☐ 打开HTML ☐ 打开UBB ☒ 打开表情  ☐ 隐藏 ☐ 记住我 [\[登入\]](#) [\[注册\]](#)

提交

重置